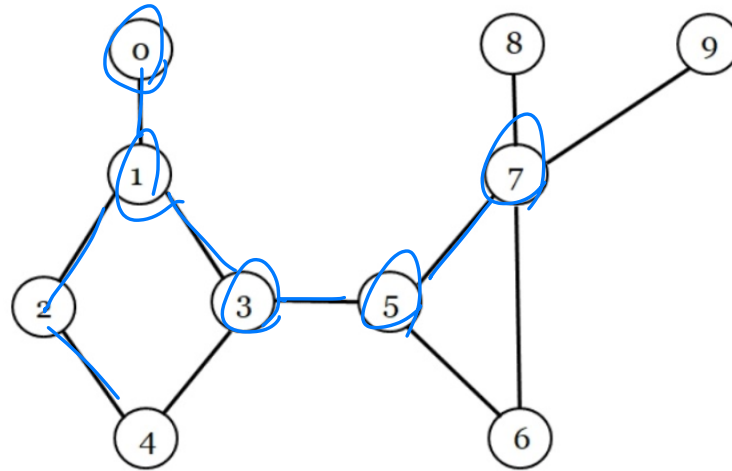


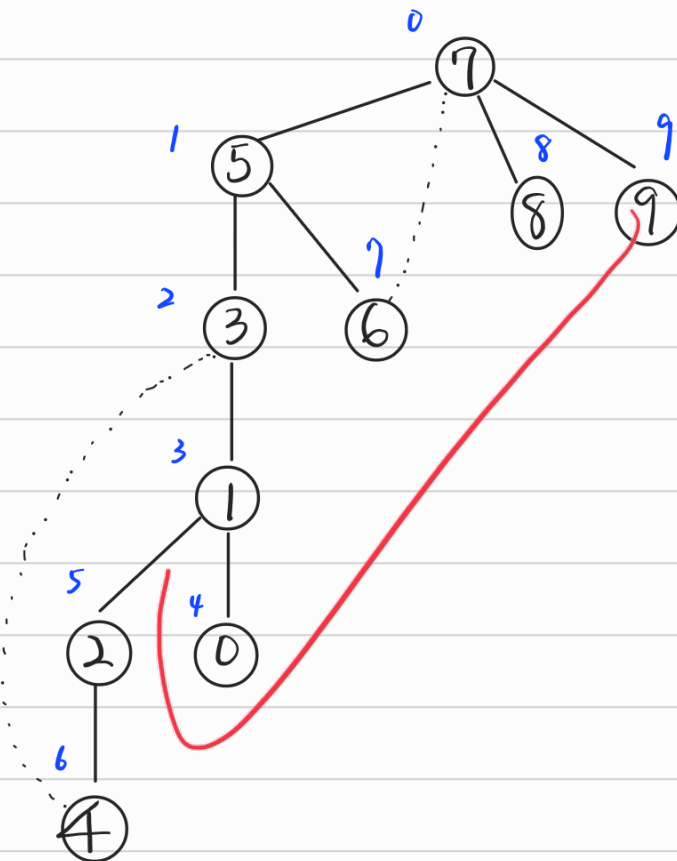
1. (20%) Use Depth-First-Search from node 7 and follow the alphabetical order to explore the next vertex (i.e., 先走數字最小的相鄰節點).

- (a) Redraw the connected graph with back edge
 (b) Show the **dfn** and **low** values of each node.



100

(a) DFS $\rightarrow 7, 5, 3, 1, 0, 2, 4, 6, 8, 9$



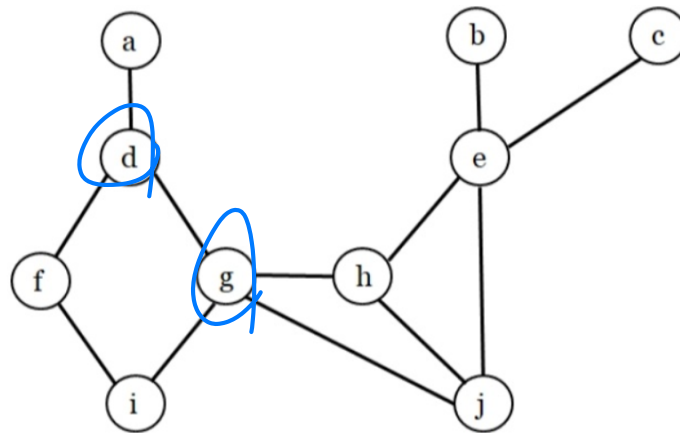
(b)

v	0	1	2	3	4	5	6	7	8	9
dfn	4	3	5	2	6	1	7	0	8	9
low	4	2	2	2	2	0	0	0	8	9

2. (10%) Given the following graph

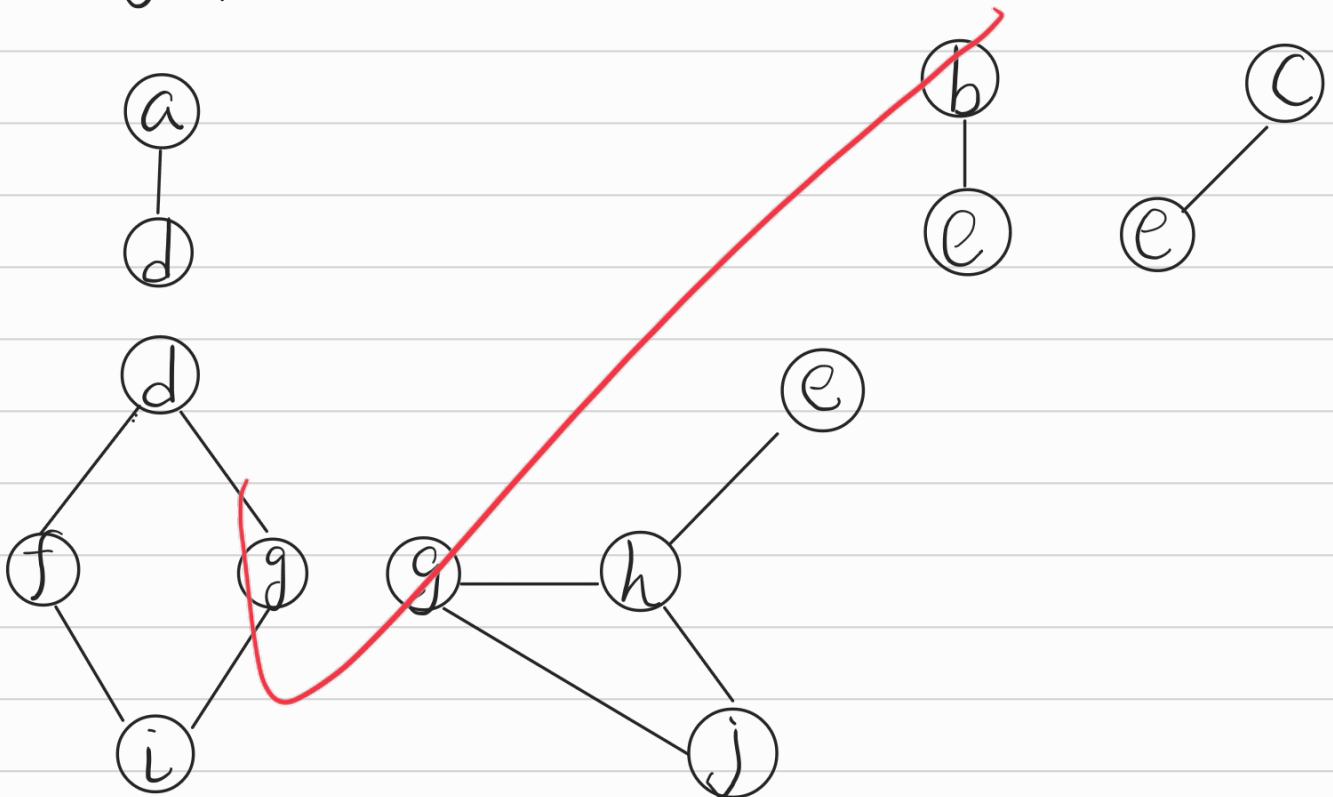
(a) Find all of the articulation points.

(b) Find all of the biconnected components.



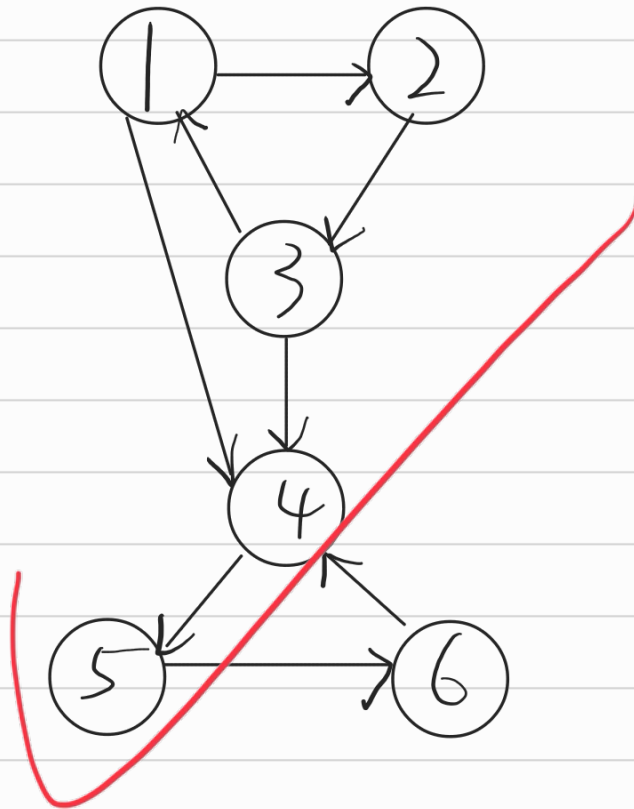
(a) d, g, e

(b)



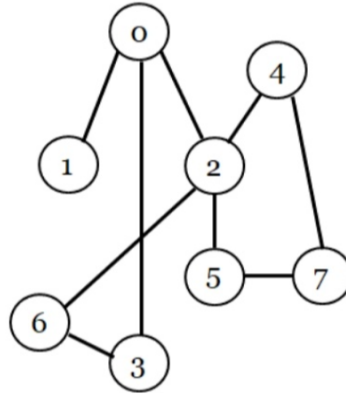
3. (10%) Draw a graph that has A as its adjacency matrix. The vertices of the graph are labeled from 1 to 6.

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

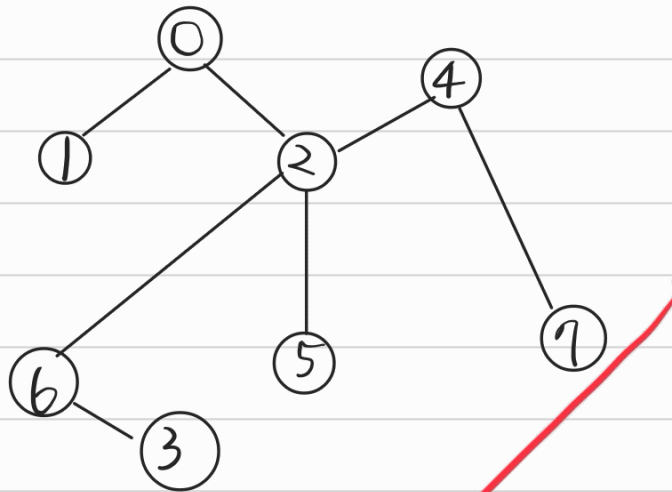


4. (10%) Consider the following undirected graph.

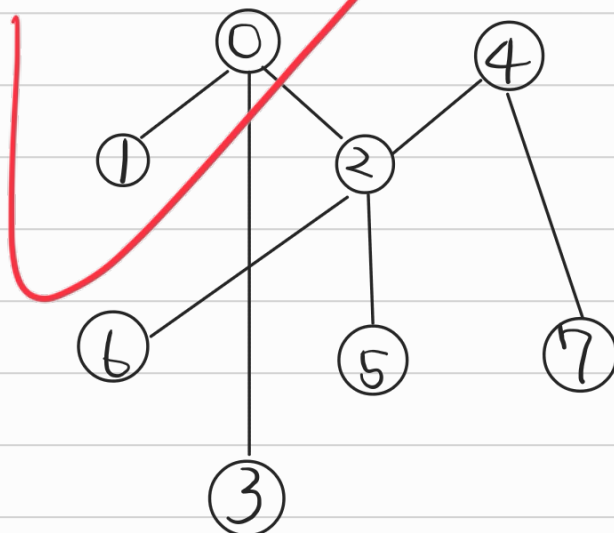
- (a) Write the depth first spanning tree starting at node 0 (**visit the smaller number first**)
- (b) Write the breadth first spanning tree starting at node 0 (**visit the smaller number first**)



(a) 0 1 2 4 7 5 6 3



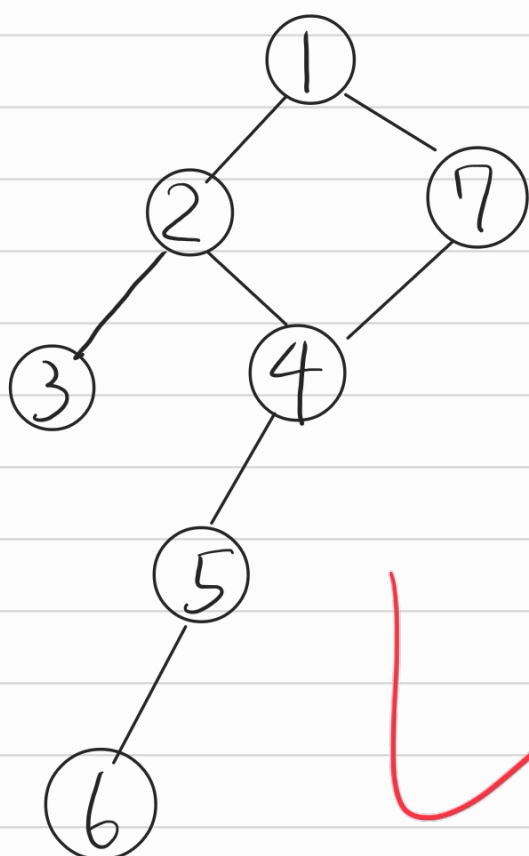
(b) 0 1 2 3 4 5 6 7



5. (10%) Let $G = (V, E)$ be an undirected graph with adjacency matrix A as shown below. The vertices of the graph are labeled from v_1 to v_7 . If the starting vertex is v_1 .

- (a) Use a breadth-first search based on A to determine the visiting order of G .
 (b) Use a depth-first search based on A to determine the visiting order of G .

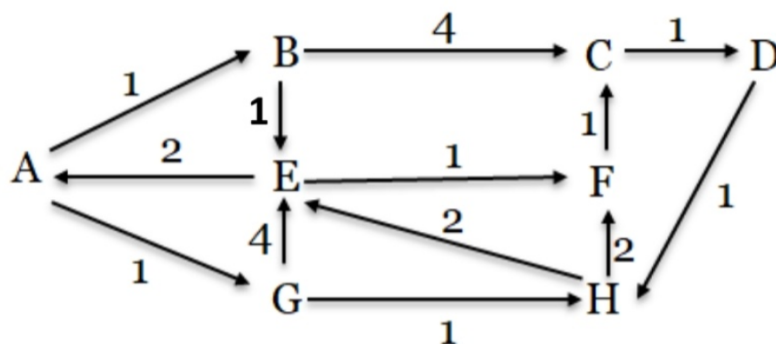
$$A = \begin{matrix} \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$



(a) $V_1, V_2, V_7, V_3, V_4, V_5, V_6$

(b) $V_1, V_2, V_3, V_4, V_5, V_6, V_7$

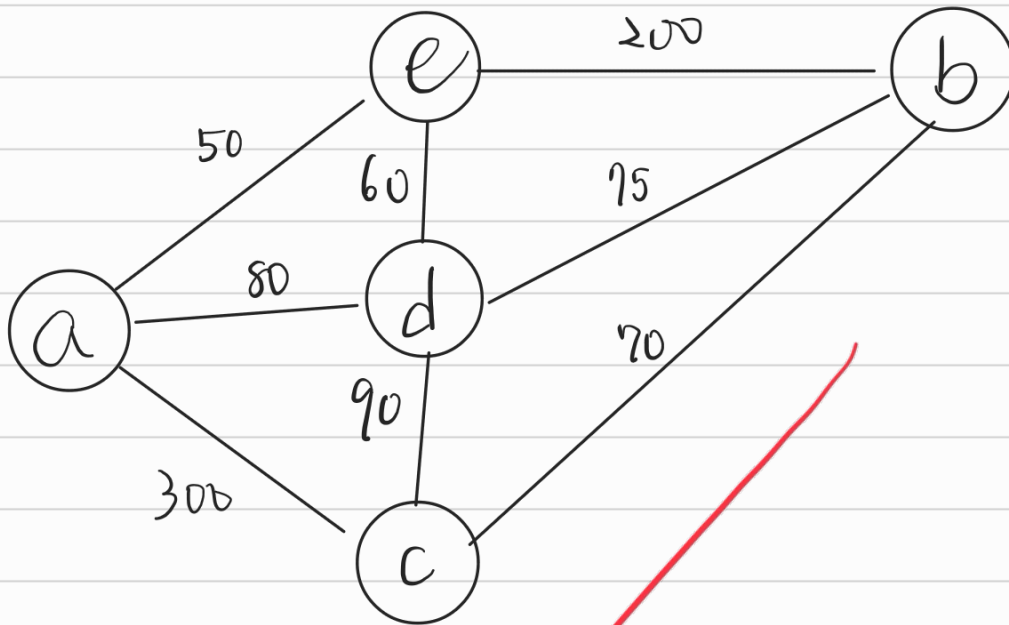
6. (10%) A weighted directed graph G is given as follows. Filling in the given table using Dijkstra's algorithm to find shortest path from A to all nodes. If more than 1 vertices have the same minimum distance, choose the vertex with alphabet priority.



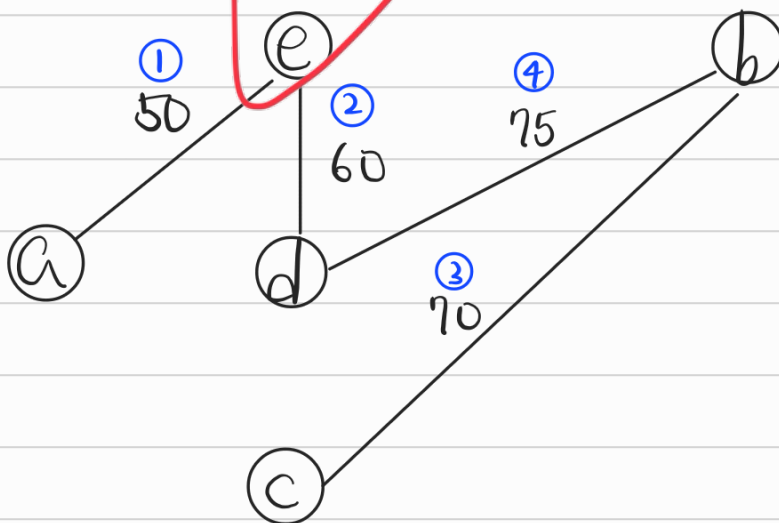
Iteration	Vertex Selected	Distance							
		A	B	C	D	E	F	G	H
Initial	A	0	vis	vis	vis	vis	vis	vis	vis
1	B	0	1	∞	∞	2	∞	1	∞
2	G	0	1	5	∞	2	∞	1	2
3	E	0	1	5	∞	2	3	1	2

4	H	0	1	5	∞	2	3	1	2
5	F	0	1	4	∞	2	3	1	2
6	C	0	1	4	5	2	3	1	2
7	D	0	1	4	5	2	3	1	2

7. (15%) Suppose you are given an undirected graph with five nodes a, b, c, d and e . Also the graph has eight edges $e_{ac}, e_{ad}, e_{ae}, e_{bc}, e_{bd}, e_{eb}, e_{cd}, e_{de}$ with weights 300, 80, 50, 70, 75, 200, 90 and 60, respectively, draw the minimum-cost spanning tree for this graph and find the total cost of the tree under
- (a) Kruskal's algorithm
 - (b) Prim's algorithm (initial node: a)
 - (c) Sollin's algorithm

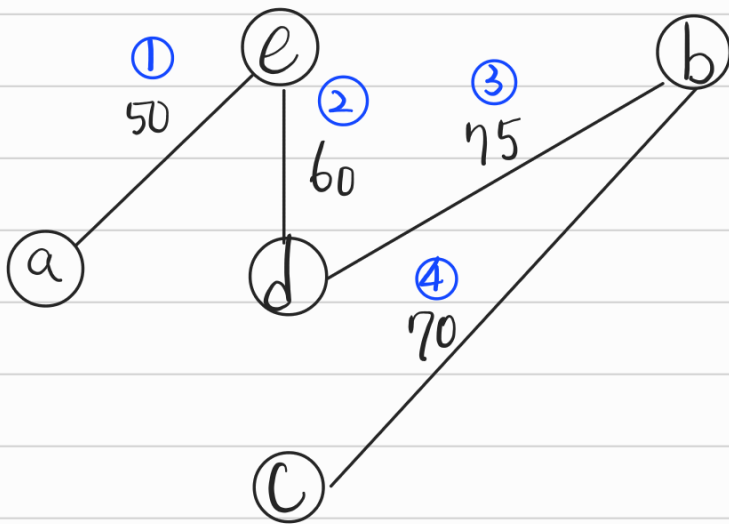


(a). Kruskal's



cost = 255 #

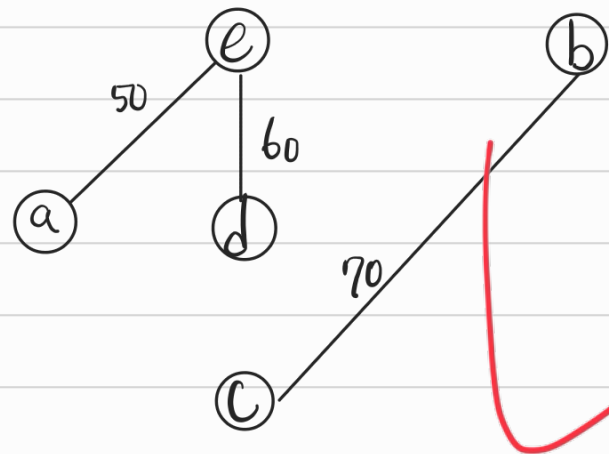
(b) Prim's



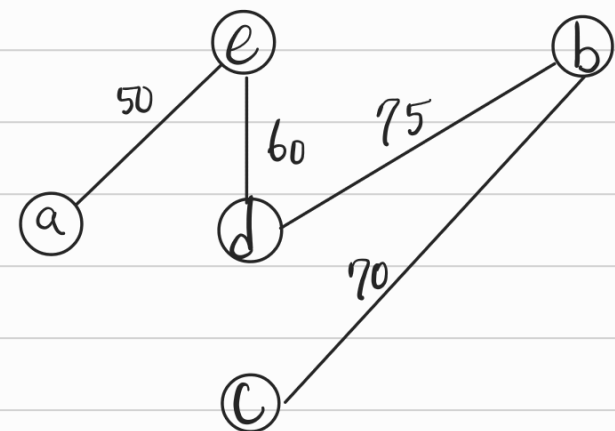
cost = 255 #

(c) Sollin's

stage 1: $(a, e), (b, c), (c, b), (d, e), (e, a)$
remove duplicate: $(a, e), (b, c), (d, e)$

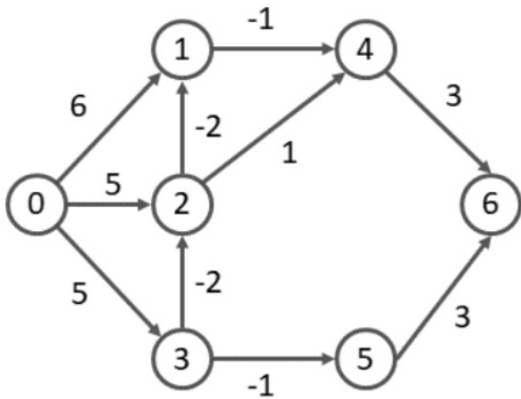


stage 2: (d, b) # $n-1$ edges



cost = 255 #

8. (15%) A weighted directed graph G is given as follows. Filling in the given table using Bellman-Ford algorithm to find shortest path from 0 to all nodes.



k	$dist^k[u]$						
	0	1	2	3	4	5	6
1	0	6	5	5	∞	∞	∞
2	0	3	3	5	5	4	∞
3	0	1	3	5	2	4	7
4	0	1	3	5	0	4	5
5	0	1	3	5	0	4	3
6	0	1	3	5	0	4	3